

COMP3211

05. System Modelling

Dr. Max Yu Pei

Intended Learning Outcomes

- ❖ Upon completion of this lecture, you will
 1. understand how graphical models can be used to represent software systems and why several types of model are needed to fully represent a system;
 2. understand the fundamental system modeling perspectives of context, interaction, structure, and behavior;
 3. understand the principal diagram types in the Unified Modeling Language (UML) and how these diagrams may be used in system modeling;
 4. understand model-driven engineering, where an executable system is automatically generated from structural and behavioral models.

System Modeling

- ❖ System modeling is the process of developing abstract models of a system
 - A model is an *abstraction* of the system being studied rather than an alternative *representation* of that system.
- ❖ Models can be
 - from different perspectives (e.g., external vs. internal, structural vs. behavioral)
 - in different forms (e.g., graphical vs. formal/mathematical)
 - for systems at different stages (e.g., existing vs. to be developed)

Unified Modeling Language (UML)

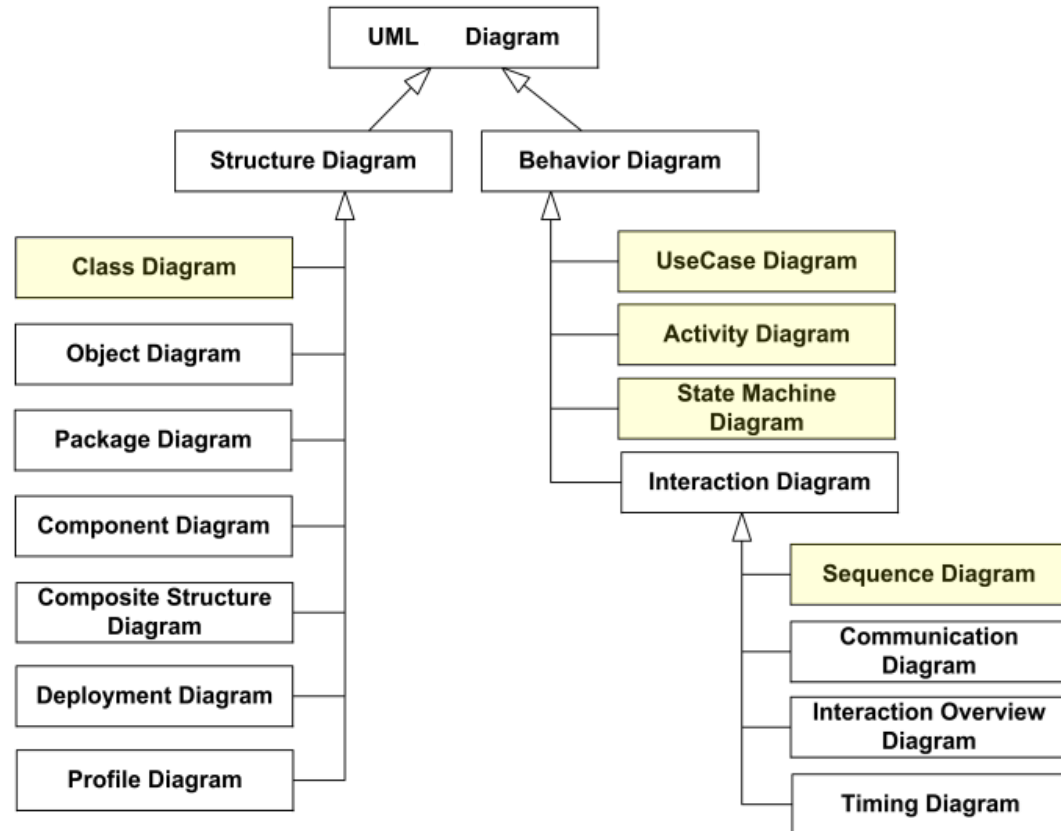


- ❖ A standardized language for *specifying, visualizing, constructing, and documenting* (software) systems.
 - General purpose modeling language (for [OO software] systems)
 - Today's de-facto standard in the industry

- ❖ Usage
 - To stimulate and focus discussion about a system
 - Incomplete and informal (normal in agile modeling)
 - To document an existing system
 - Correct but incomplete
 - To generate a system implementation
 - Complete and correct

UML Diagram Types

- ❖ 14 types of diagrams: 7 structure + 7 behavior
 - A UML model consists of diagrams and documentation that complements the diagrams
 - Different diagrams provide different levels of abstraction
 - High-level vs. low-level
 - Component vs. Object
 - Use-case vs. Sequence
- ❖ Most users of the UML thought that five diagram types could represent the essentials of a system
 - Erickson and Siau 2007



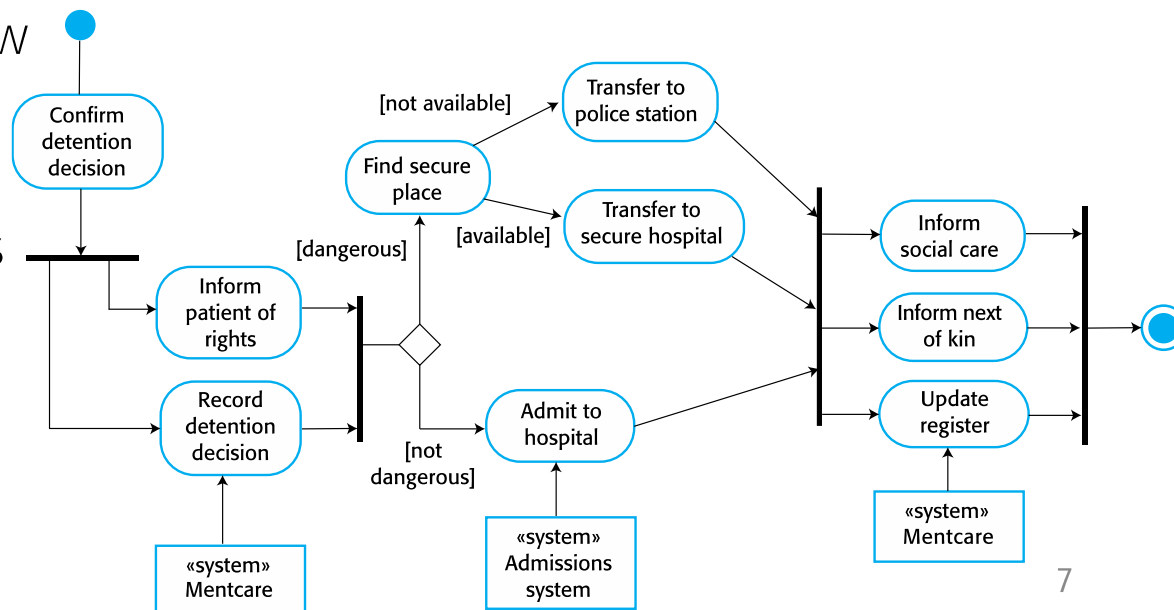
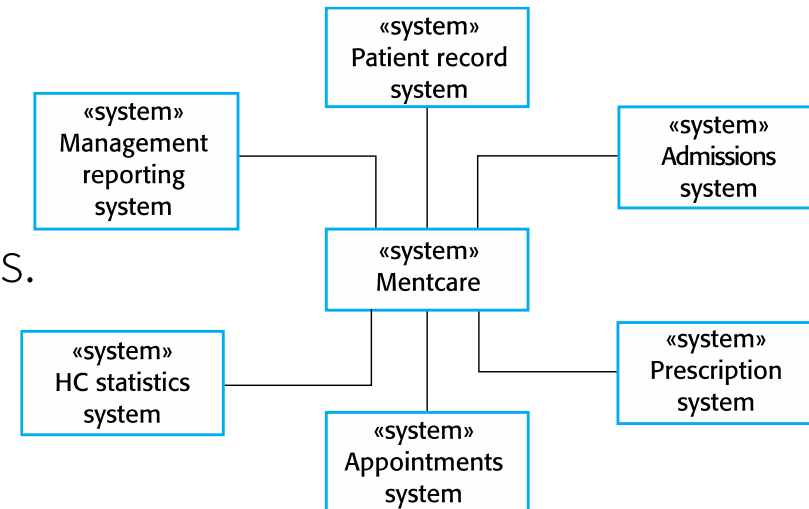
CONTEXT MODELS

Context Models

❖ Context models illustrate a system's operational context

➤ Operational context defines how a system interacts with its environment, without detailing internal implementation mechanics.

➤ Business process models defined using UML activity diagrams may help reveal how the system being developed is used in broader business processes.



STRUCTURAL MODELS

Structural Models

- ❖ Display the organization of a system in terms of the components that make up that system and their relationships.
 - Static vs. dynamic
 - Overall vs. focused
- ❖ Class Diagrams illustrate the static structure of an object-oriented system by showing its classes and their relationships.
 - Class: A template or data type defining a specific kind of system object; individual data entries are instantiated as objects of that class.
 - Usage: Applied across the analysis (domain modeling), design (system structure), and implementation (code architecture) phases of software development.

Classes and Objects

- ❖ Class ~ Object vs. Type ~ Variable (e.g., `int x`)
- ❖ A class is the structural blueprint, while an object is a specific instance created from that blueprint.
 - Class (Blueprint): Defines the abstract data type, specifying what attributes (state) and methods (behavior) any created object will possess.
 - Object (Instance): The concrete, runtime entity allocated in memory containing real data values for those defined attributes.
- ❖ Examples
 - Watching television: Both the remote and the television are objects with properties and behaviors
 - `BigDecimal`: Has digits and can do addition, subtraction, multiplication, and division

Classes

- ❖ A class encapsulates state (attributes) and behavior (operations)

Octopus
- amount: float - ID: string
+ getAmount(): int + getID(): string + withdraw(int)

Name

Attributes

Operations

-: private
+: public
#: protected
/: derived
Underlined: static

- Each attribute has a type
 - Each operation has a signature
- ❖ The class name is the only mandatory information

Octopus
amount ID
getAmount() getID()

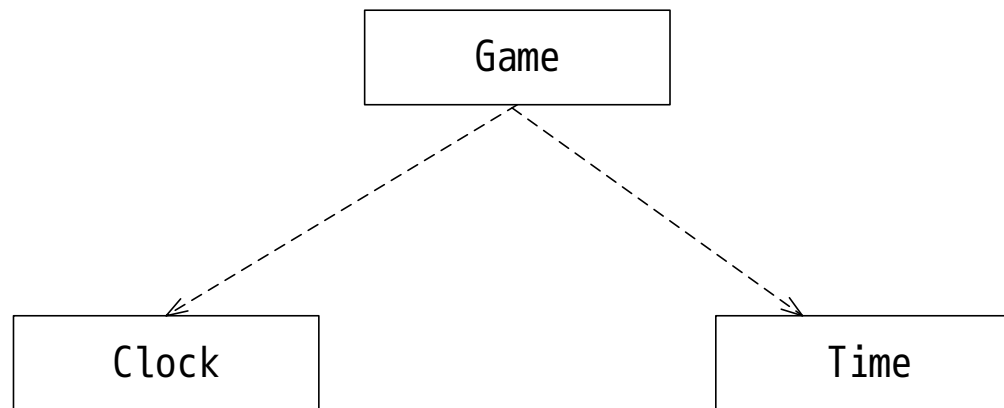
Octopus

Relationships

- ❖ General relationship
 - Dependency
- ❖ Instance-level (objects are also called instances)
 - Association
 - Aggregation
 - Composition
- ❖ Class-level
 - Generalization
 - Realization/Implementation

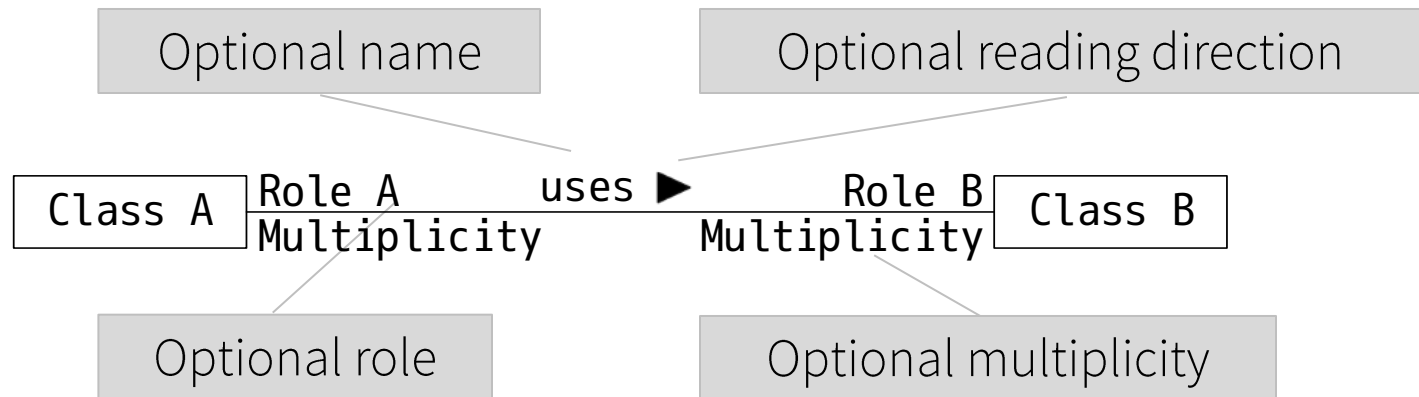
Dependency

- ❖ A semantic connection between dependent and independent classes (modules).
 - If changes to the definition of one class (module) (the server or target) may cause changes to the other (the client or source).



Associations

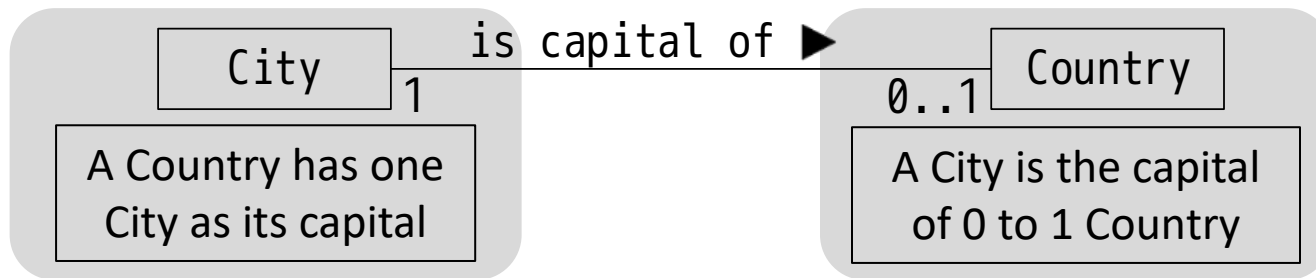
- ❖ An association is a type of relation between class instances
- ❖ Objects of the classes can communicate using the association, e.g.
 - Class A has an attribute of type B
 - Class A creates instances of B
 - Class A received a message with argument of type B



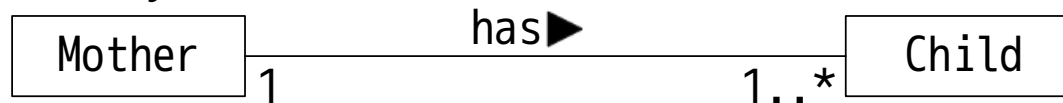
Association Multiplicity

- ❖ Multiplicity denotes how many objects of the class take part in the relation

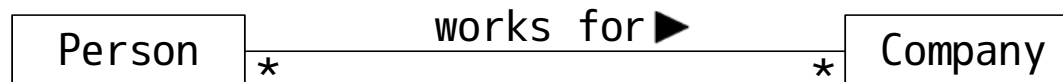
- 1-to-1



- 1-to-many

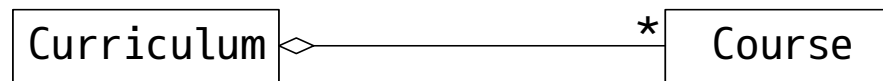


- Many-to-many

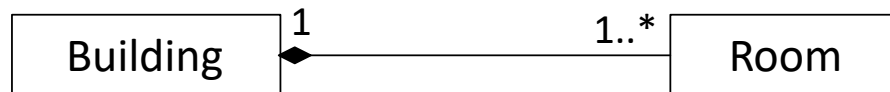


Special Associations (Has-A Relation)

- ❖ Aggregation: “part-of” relation between objects
 - Component can be part of multiple aggregates
 - Component can be created and destroyed independently of the aggregate

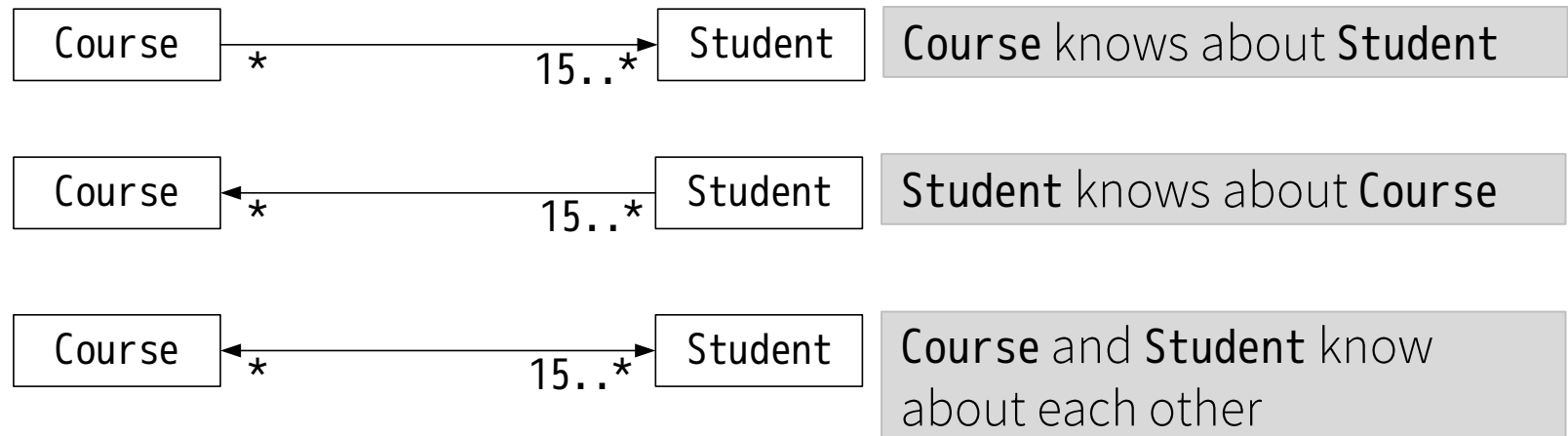


- ❖ Composition: strong aggregation
 - A component can only be part of a single aggregate
 - Exists only together with the aggregate



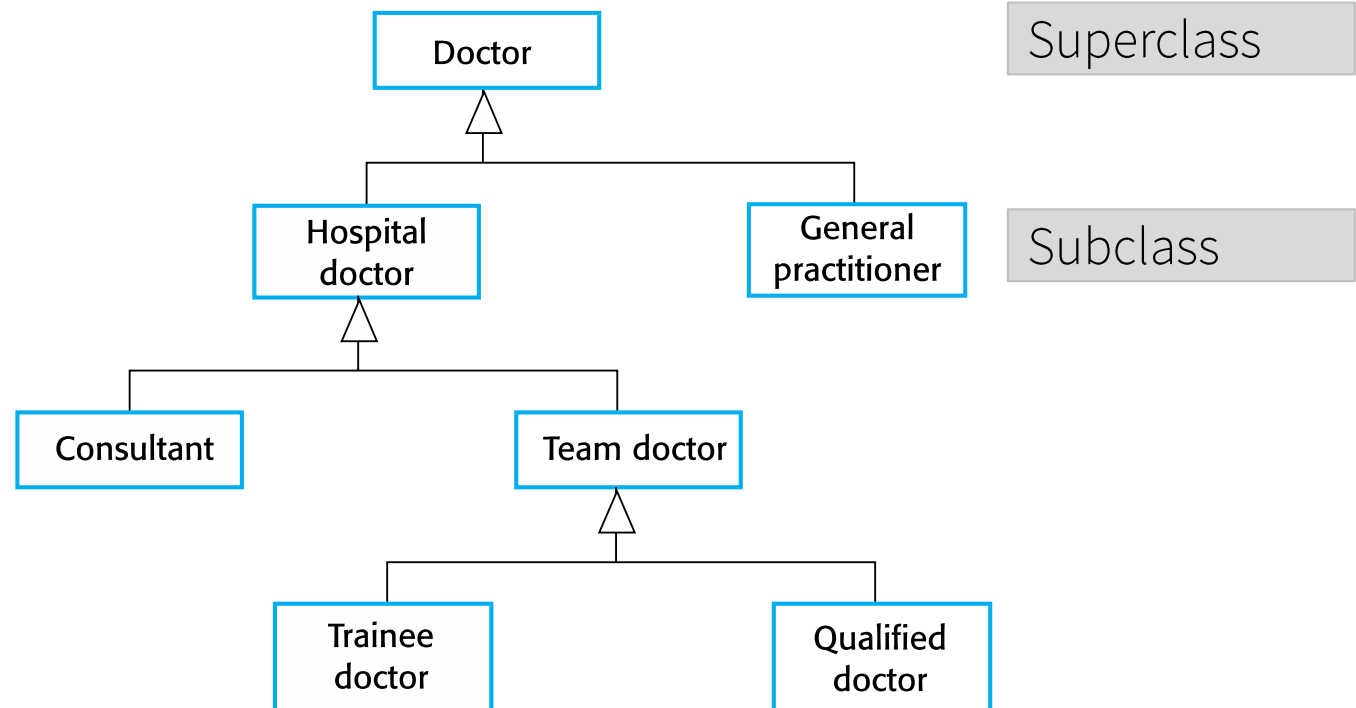
Navigability of Association

- ❖ Association can be directed
 - Direction denotes whether objects can be accessed through this association

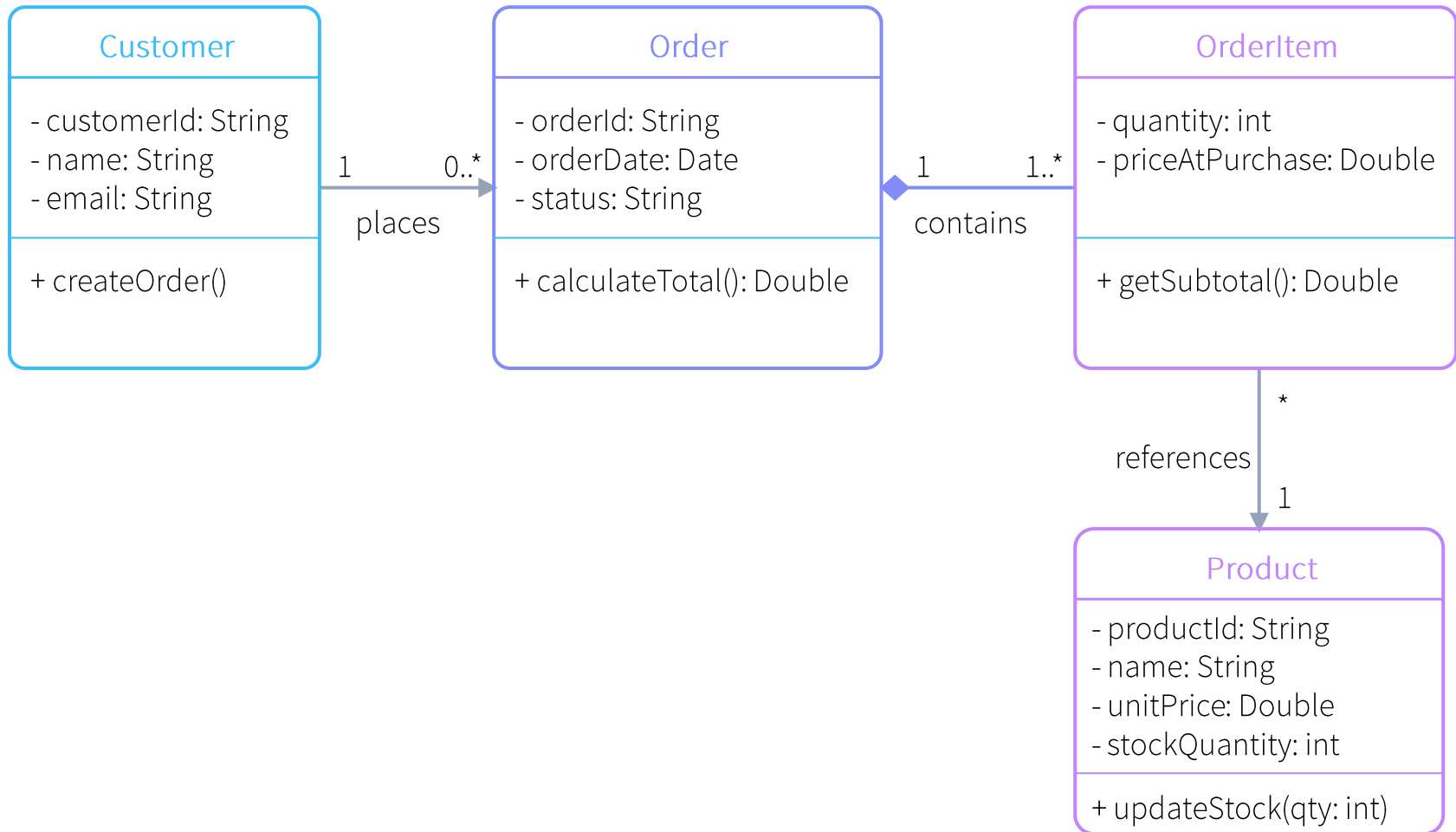


Generalization and Specialization

- ❖ Generalization relation expresses a kind-of (“Is-A”) relationship
- ❖ Generalization is implemented by inheritance
 - The child classes inherit the attributes and operations of the parent class
- ❖ Generalization simplifies the model by eliminating redundancy



Example Class Diagram



INTERACTION MODELS

Interaction Models

- ❖ All systems involve interactions of some kind
 - Modeling user interaction helps to identify user requirements.
 - Modeling system-to-system interaction highlights the communication problems that may arise.
 - Modeling component interaction helps us understand if a proposed system structure will deliver the required performance and dependability.
- ❖ Two related approaches to interaction modelling
 - Use case diagrams model interactions between systems and external agents (human users or other systems);
 - Sequence diagrams model interactions between system components (and external agents)

Use Case Modelling

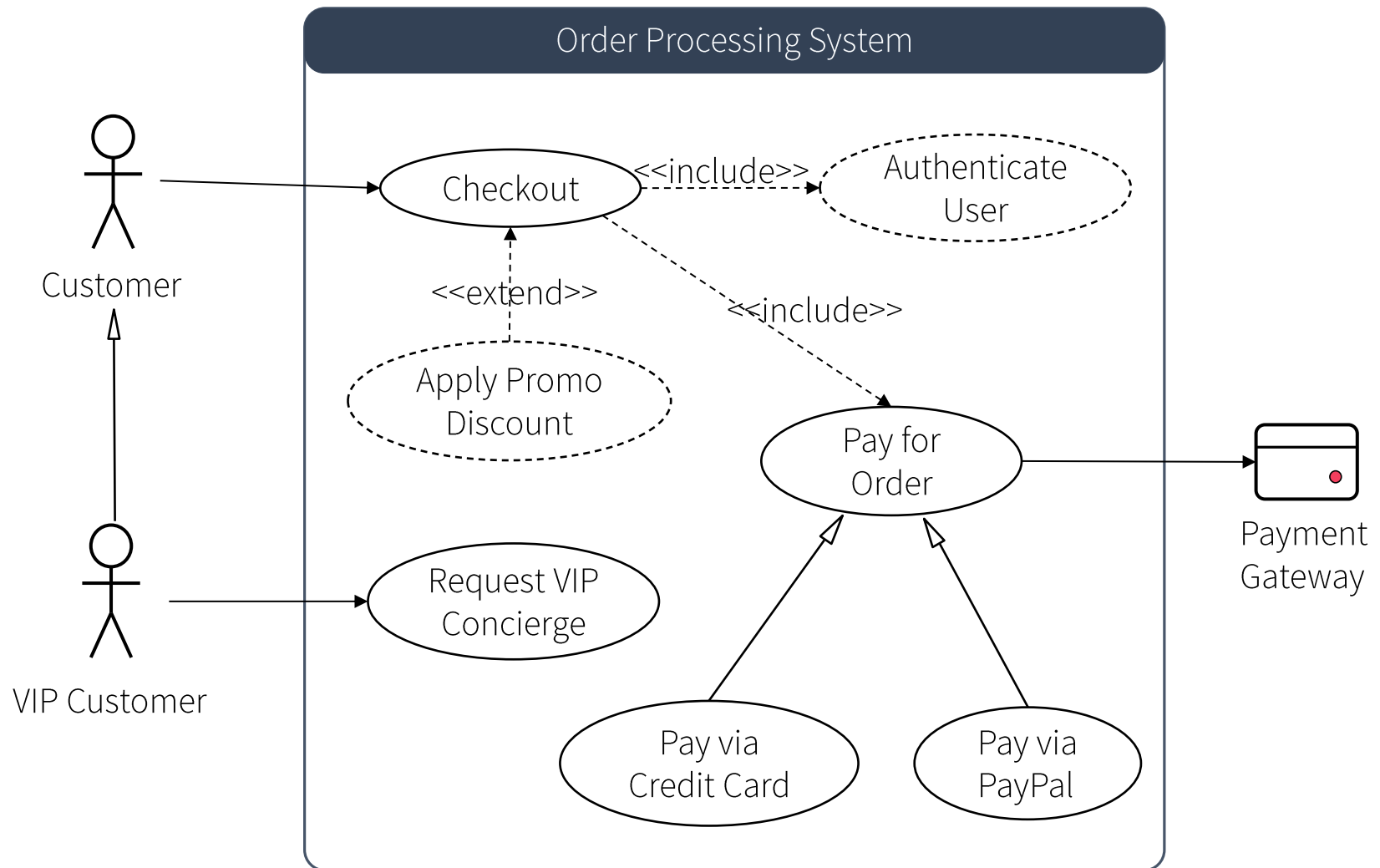
- ❖ Each use case (an ellipse) represents a discrete task that involves external interaction with a system.
- ❖ An actor (a stick figure) in a use case may be a person or a system.
- ❖ Relationships

Relationship Type	Applied Between	Arrow Style	Purpose
Association	Actor ↔ Use Case	Solid line	Connecting an actor to the interactions they perform.
Generalization	Use Case → Use Case	Solid line with hollow triangle	Specializing generic behavior (polymorphism/inheritance).
Actor Generalization	Actor → Actor	Solid line with hollow triangle	Creating specialized roles/permissions for actors.
Include	Use Case → Use Case	Dashed with <<include>>	Reusing common mandatory execution steps.
Extend	Use Case → Use Case	Dashed with <<extend>>	Adding optional or conditional steps to a base flow.

<<include>> vs. <<extend>>

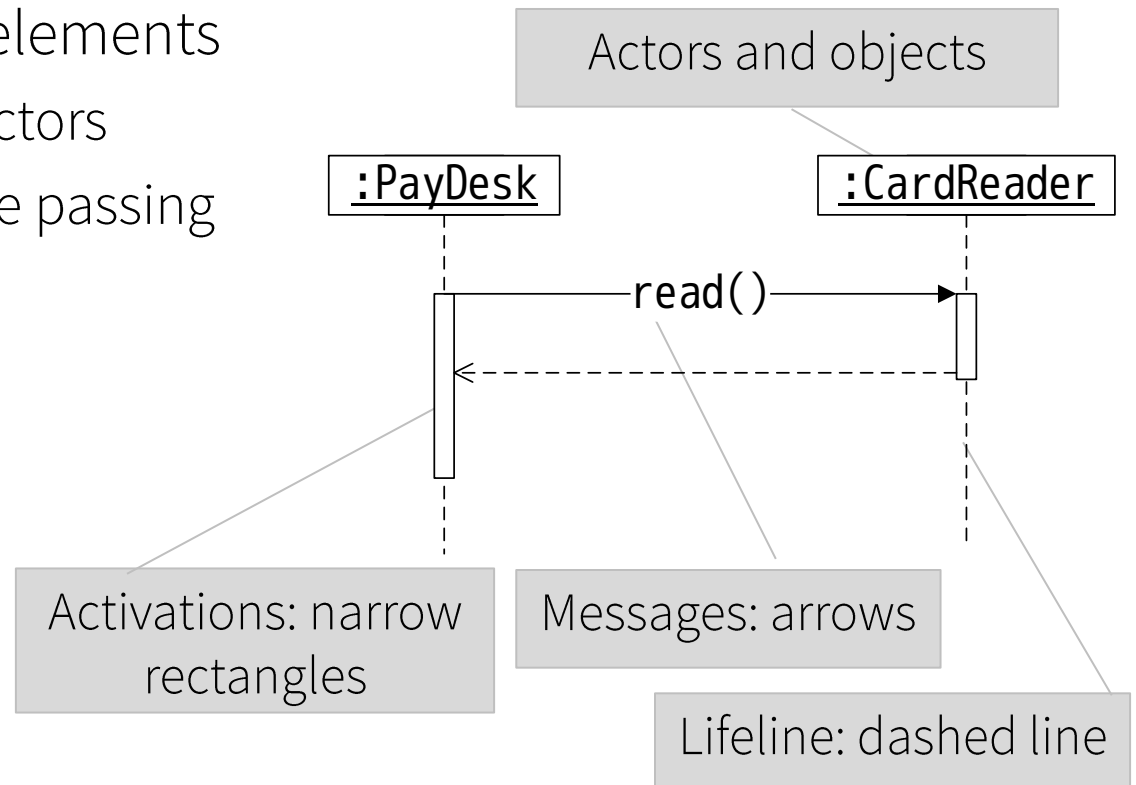
- ❖ Include: A base use case requires another use case to complete its operation. It is used to factor out common step sequences shared across multiple use cases into a reusable unit.
 - Base Use Cases: Withdraw Cash and Check Balance.
 - Included Use Case: Authenticate User.
- ❖ Extend: A use case optionally adds behavior to a base use case under specific conditions (defined at an extension point). The base use case can complete successfully on its own without ever triggering the extension.
 - Base Use Case: Withdraw Cash.
 - Extension Use Case: Send Low Balance Alert.
 - Extension Condition: Triggered only if the account balance falls below a threshold after withdrawal.

Example: Order Processing System

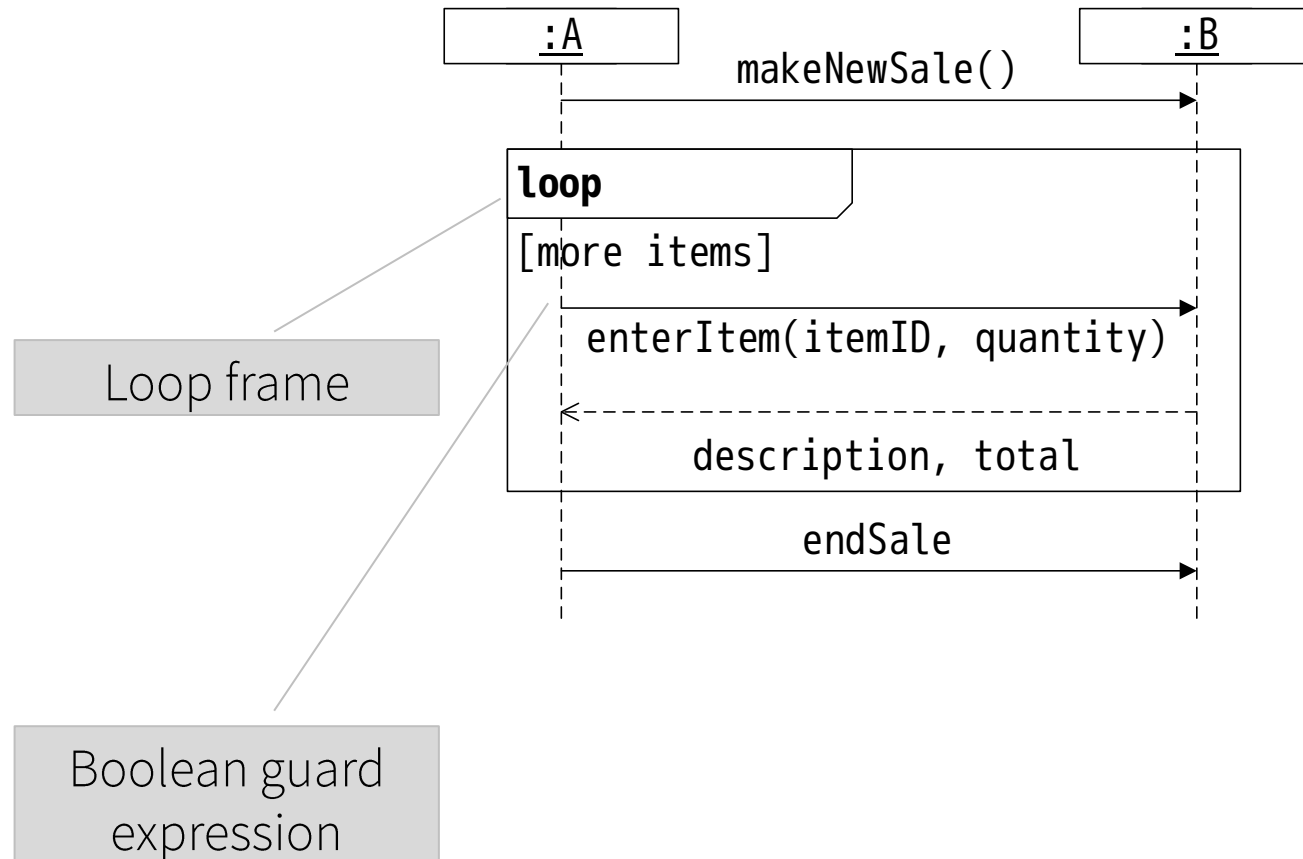


Sequence Diagrams

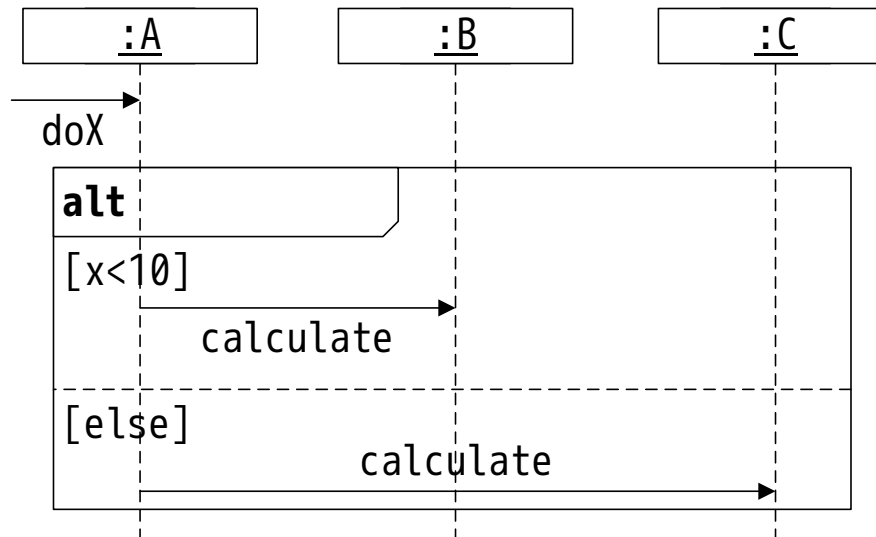
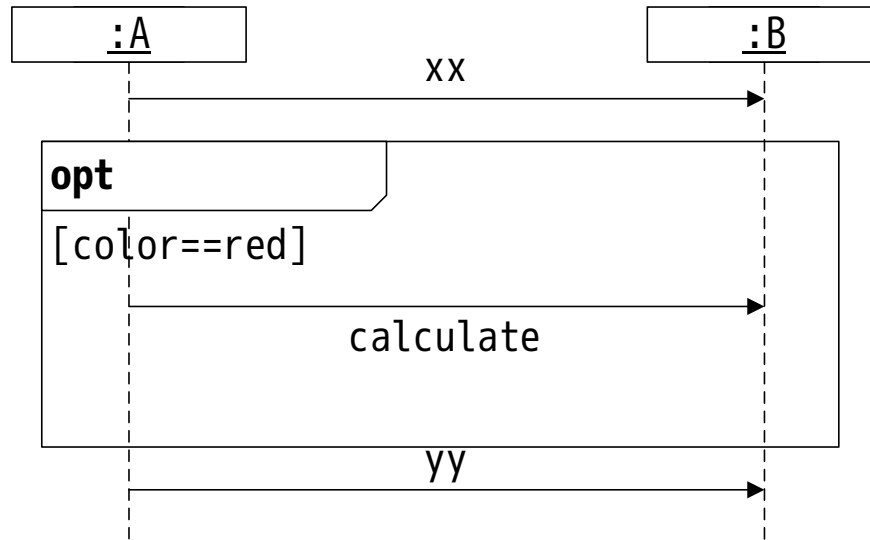
- ❖ To model the interactions between the actors and the objects within a system and those between the objects themselves.
 - A sequence diagram shows the sequence of interactions during a particular process or process instance.
- ❖ Sequence diagram elements
 - Entities: objects/actors
 - Relations: message passing
 - Syntactic sugar:
 - lifelines
 - activations
 - frames



Frame for Looping



Frame for Conditioning

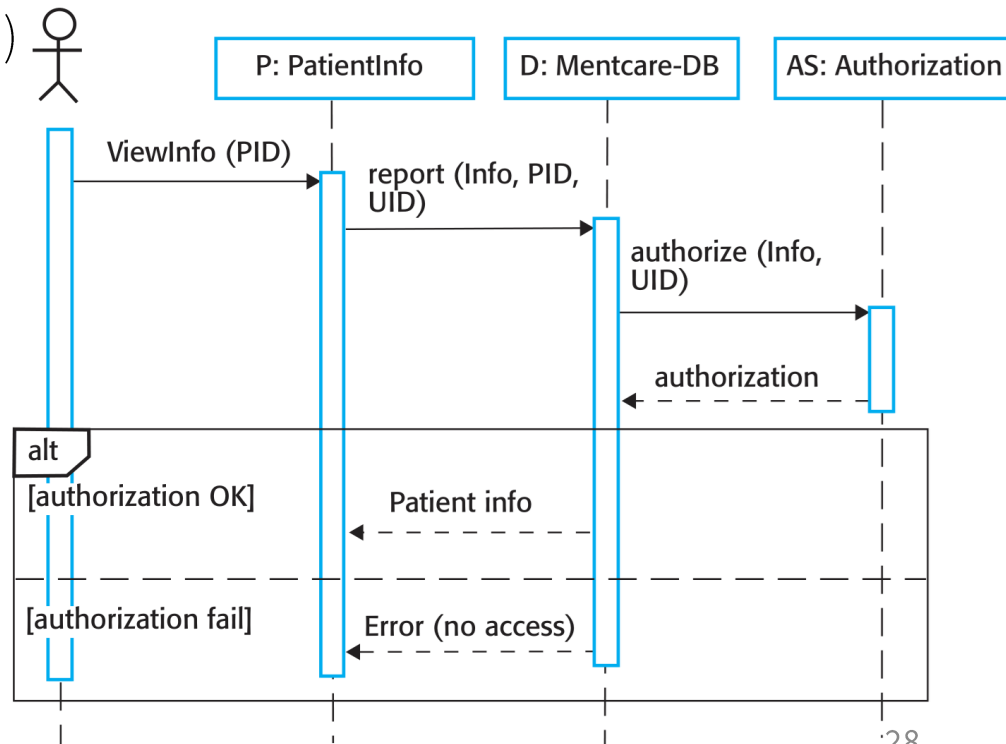


Frames could
be nested.

Example: Viewing Patient Information

1. The medical receptionist triggers the ViewInfo method in an instance P of the PatientInfo object class, supplying the patient's identifier, PID to identify the required information. P is a user interface object, which is displayed as a form showing patient information.
2. The instance P calls the database to return the information required, supplying the receptionist's identifier to allow security checking. (At this stage, it is not important where the receptionist's UID comes from.)
3. The database checks with an authorization system that the receptionist is authorized for this action.
4. If authorized, the patient information is returned and is displayed on a form on the user's screen. If authorization fails, then an error message is returned.

Medical Receptionist



BEHAVIORAL MODELS

Behavioral Models

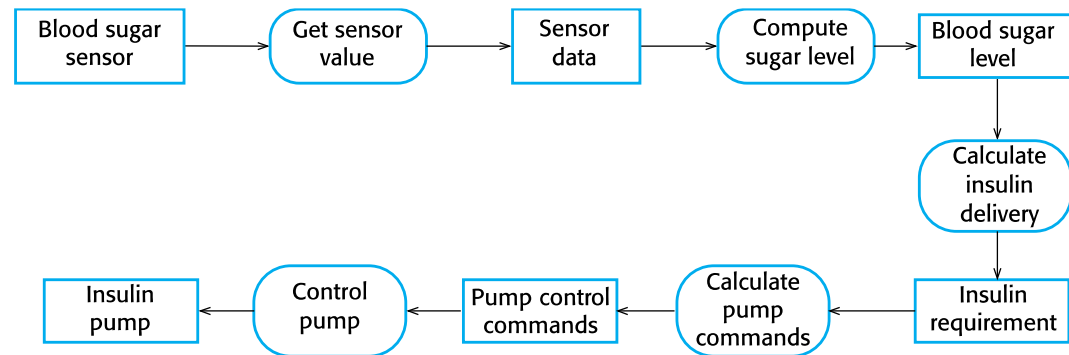
- ❖ Behavioral models model a system's dynamic behavior as it executes.
 - They show what happens or what is supposed to happen when a system responds to a stimulus from its environment.
 - You can think of these stimuli as being of two types:
 - Data. Some data arrives and must be processed by the system. The availability of the data triggers the processing.
 - Events. Some event happens that triggers system processing. Events may have associated data, although this is not always the case.

Data-Driven Modeling

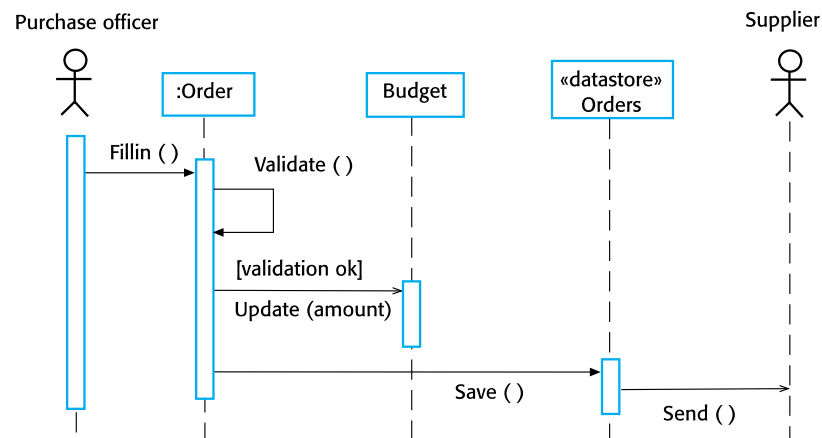
- ❖ Data-driven models show the sequence of actions involved in processing input data and generating an associated output.
 - Many business systems are data-processing systems that are primarily driven by data.

- ❖ Examples

- Using activity diagrams



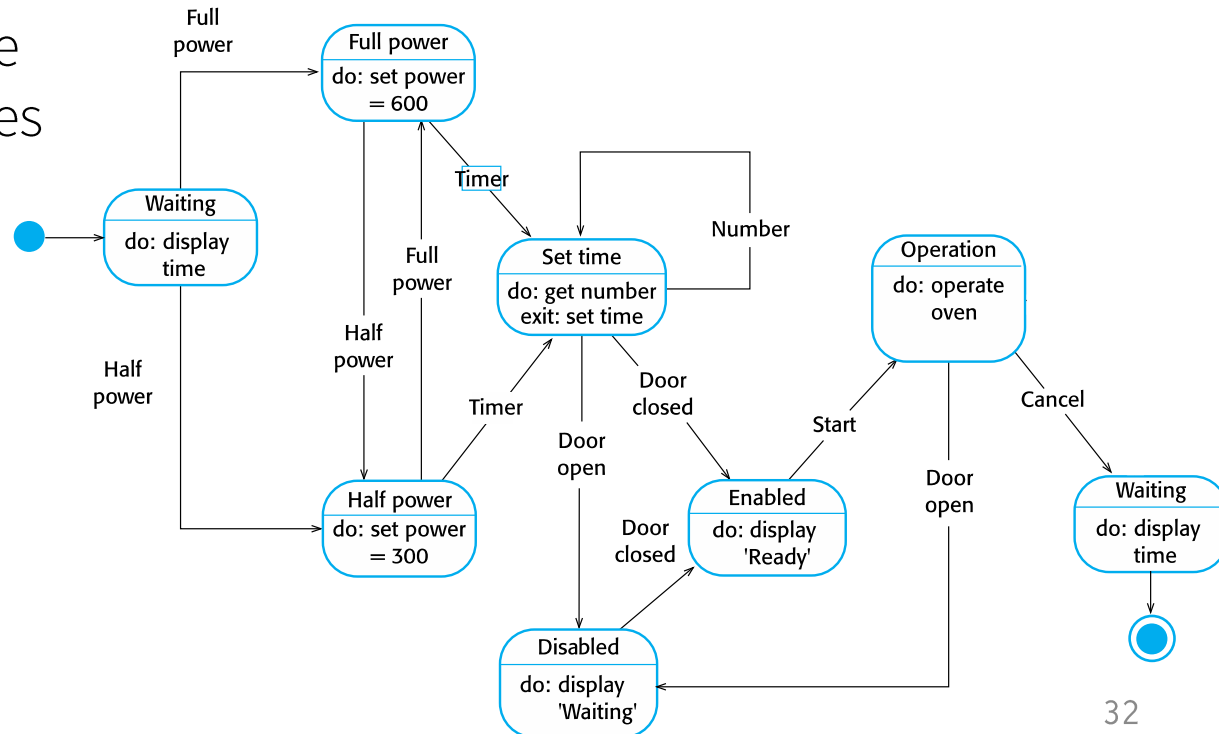
- Using sequence diagrams



Event-Driven Modeling

- ❖ Event-driven modeling shows how a system responds to external and internal events.
 - It assumes that a system has a finite number of states and that events (stimuli) may cause transitions between states.
- ❖ UML state diagrams show system states and events that cause transitions from one state to another.

- System states are modeled as nodes and events as arcs between the nodes.



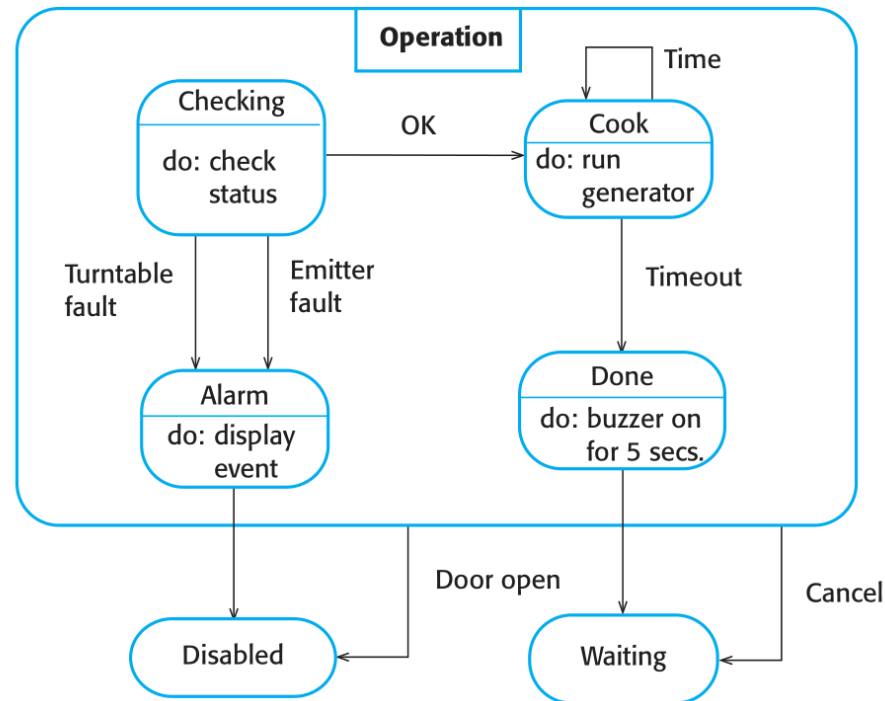
States and Stimuli for The Microwave Oven

State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for five seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding.

Stimulus	Description
Half power	The user has pressed the half-power button.
Full power	The user has pressed the full-power button.
Timer	The user has pressed one of the timer buttons.
Number	The user has pressed a numeric key.
Door open	The oven door switch is not closed.
Door closed	The oven door switch is closed.
Start	The user has pressed the Start button.
Cancel	The user has pressed the Cancel button.

Superstates

- ❖ The number of possible states increases rapidly
 - Modelling states at different levels
 - A superstate looks like a single state in a high-level model but is then expanded to show more detail in a separate diagram



Key Points (1)

- ❖ A model is an abstract view of a system that ignores system details. Complementary system models can be developed to show the system's context, interactions, structure and behavior.
- ❖ Context models show how a system that is being modeled is positioned in an environment with other systems and processes.
- ❖ Use case diagrams and sequence diagrams are used to describe the interactions between users and systems in the system being designed. Use cases describe interactions between a system and external actors; sequence diagrams add more information to these by showing interactions between system objects.
- ❖ Structural models show the organization and architecture of a system. Class diagrams are used to define the static structure of classes in a system and their associations.

Key Points (2)

- ❖ Behavioral models are used to describe the dynamic behavior of an executing system. This behavior can be modeled from the perspective of the data processed by the system, or by the events that stimulate responses from a system.
- ❖ Activity diagrams may be used to model the processing of data, where each activity represents one process step.
- ❖ State diagrams are used to model a system's behavior in response to internal or external events.

THE END